

Rapport de Projet de Fin d'Année

ShopAI

Plateforme e-commerce enrichie par l'Intelligence Artificielle

Master 1 CDA — 4ème Année
Application Full-Stack Web & Mobile (PWA)

Équipe : 2 étudiants
Année universitaire 2024 — 2025

Sommaire

1. Introduction et contexte

Ce rapport présente ShopAI, une application e-commerce full-stack développée dans le cadre du projet de fin d'année du Master 1 CDA. L'objectif n'est pas de bâtir une application entièrement fondée sur l'IA, mais de concevoir une plateforme marchande complète et fonctionnelle dans laquelle un module d'Intelligence Artificielle apporte une valeur ajoutée concrète à l'utilisateur et à l'administrateur.

ShopAI propose une interface web responsive et une version mobile progressive (PWA), un back-office d'administration complet, et deux fonctionnalités d'IA : la recommandation personnalisée de produits et la génération automatique de fiches produits. Le projet applique les méthodologies de conception (MCD, MLD, UML), un développement agile en sprints, ainsi que des bonnes pratiques de sécurité, de versioning et de CI/CD.

1.1. Problématique

Comment offrir une expérience d'achat en ligne fluide, sécurisée et personnalisée, tout en dotant l'administrateur d'outils efficaces de gestion du catalogue et des commandes, sans multiplier la charge de maintenance ?

2. Objectifs et périmètre

Le projet couvre l'ensemble du parcours e-commerce : inscription, authentification, navigation dans le catalogue, gestion du panier, passage de commande avec paiement à la livraison (COD), suivi de commande, et administration. Le module IA retenu (recommandation, option C du cahier des charges) est complété par une fonctionnalité bonus de génération de fiches produits (option B).

2.1. Fonctionnalités principales

- Espace client : catalogue, fiche produit, panier persistant, checkout, historique et suivi des commandes.
- Espace administrateur : tableau de bord (statistiques, ventes, stock), CRUD produits, gestion des commandes et des utilisateurs, journal d'activité.
- Intelligence Artificielle : recommandations de produits et génération assistée de fiches produits, avec repli déterministe.
- Sécurité : JWT, bcrypt, CAPTCHA, verrouillage anti brute-force, journalisation.

3. Conception

La modélisation s'appuie sur une démarche classique : modèle conceptuel de données (MCD), modèle logique/physique (MLD/MPD), diagramme de classes et diagramme de cas d'utilisation. Les diagrammes ci-après constituent la conception cible du système.

3.1. Modèle Conceptuel de Données (MCD)

Le MCD décrit les entités métier (Utilisateur, Produit, Catégorie, Commande, Panier, Paiement, Avis, Tag, Adresse) et leurs associations, indépendamment de toute technologie.

3.2. Modèle Logique / Physique de Données (MLD / MPD)

Le MLD traduit le MCD en tables relationnelles avec clés primaires (PK), clés étrangères (FK) et contraintes d'unicité (UQ). Le MPD précise les types physiques (varchar, decimal(10,2), datetime...). Les associations N-N (Produit-Tag, Utilisateur-Rôle) sont matérialisées par des tables de jointure.

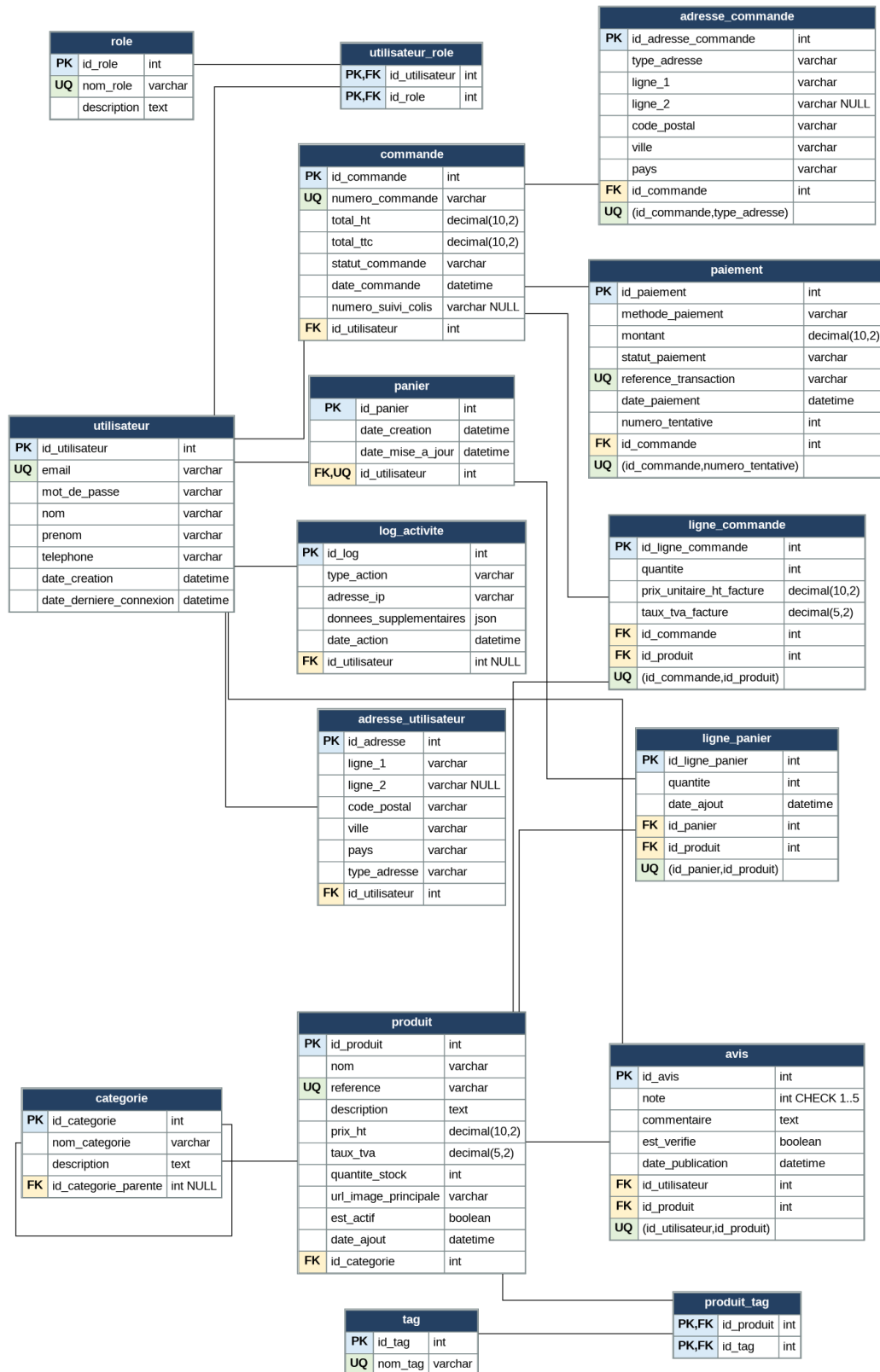


Figure 2 — Modèle Logique / Physique de Données

3.3. Diagramme de classes

Le diagramme de classes modélise le domaine objet : entités, attributs typés et services applicatifs (ServiceRecommandation, ServicePaiement) avec leurs opérations. Il fait le pont entre la conception et l'implémentation orientée services.

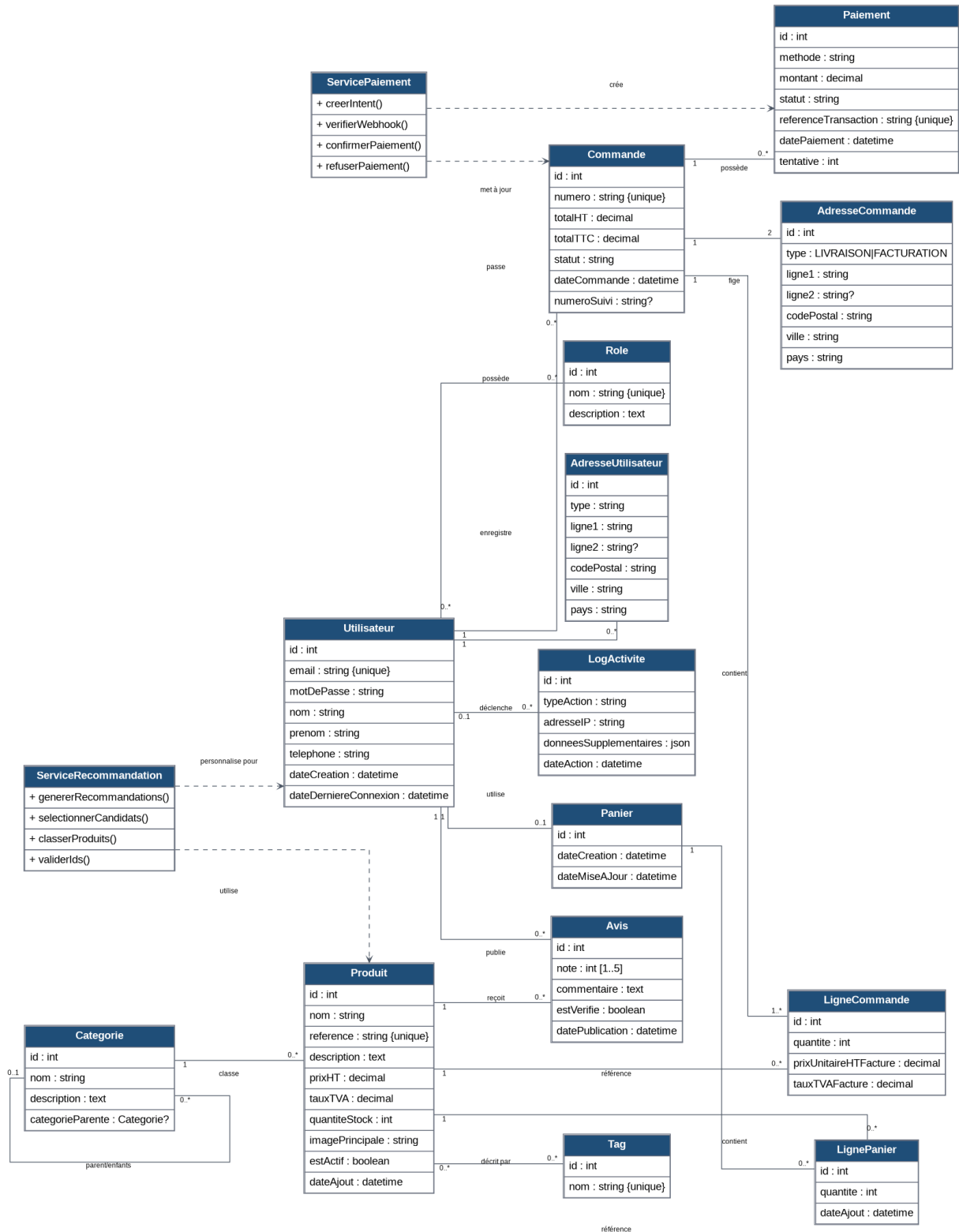


Figure 3 — Diagramme de classes

3.4. Diagramme de cas d'utilisation

Deux acteurs principaux interagissent avec la plateforme : le Client et l'Administrateur, appuyés par des acteurs secondaires (service de paiement externe, service LLM externe). Les relations « include » factorisent les comportements communs (paiement, recommandation, analyse des avis).

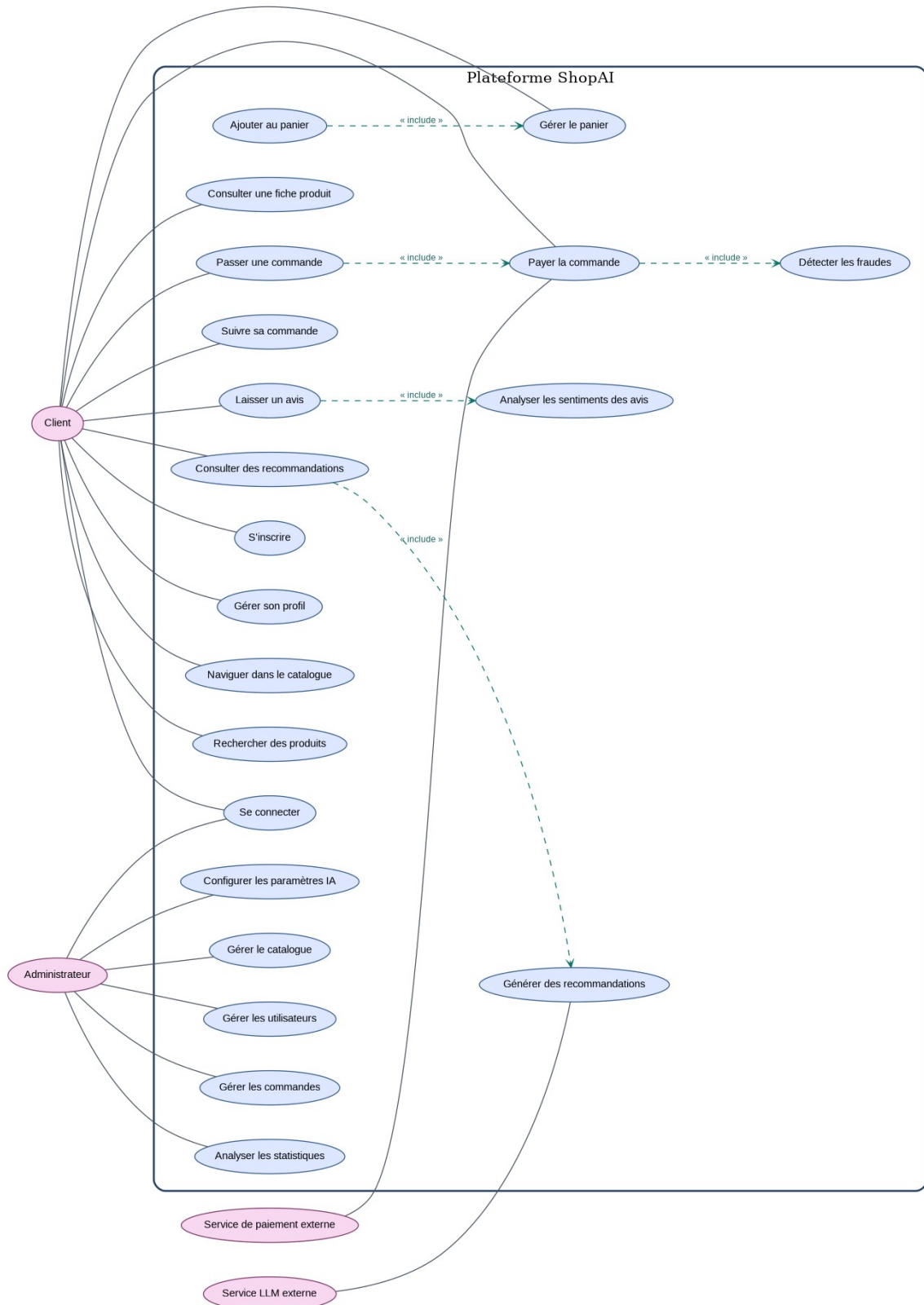


Figure 4 — Diagramme de cas d'utilisation

4. Architecture technique et choix technologiques

L'application suit une architecture en couches côté back-end : Route → Contrôleur → Service (logique métier) → Modèle (ORM) → Base de données. Des middlewares transverses assurent l'authentification, le contrôle de rôle, la limitation de débit, la journalisation et la gestion centralisée des erreurs.

4.1. Stack retenu

- Front-end : React 18 + TypeScript + Vite, Tailwind CSS, React Query (cache/état serveur), Zustand (panier), PWA.
- Back-end : Node.js + Express + TypeScript, ORM Sequelize.
- Base de données : MySQL 8 (relationnelle).
- IA : API LLM (OpenAI) avec repli déterministe si aucune clé n'est configurée.
- Qualité & Ops : Jest, GitHub Actions (CI/CD), Docker & Docker Compose, Swagger/OpenAPI.

4.2. Justification

Node.js + Express (autorisé par le cahier des charges) permet un langage unique TypeScript sur toute la stack, réduisant la charge cognitive pour une équipe de deux personnes. Sequelize apporte des requêtes paramétrées (protection contre l'injection) et un mapping objet clair. React + React Query simplifient la synchronisation avec l'API. La PWA offre une expérience mobile installable sans coût d'un développement natif séparé.

Le paiement à la livraison (COD) a été privilégié : il correspond au marché ciblé et évite la dépendance à un prestataire de paiement pour la démonstration, tout en laissant la porte ouverte à une intégration ultérieure.

5. Réalisation

5.1. Authentification et sécurité d'accès

L'inscription valide la complexité du mot de passe puis le hache avec bcrypt. La connexion impose un CAPTCHA (défi serveur à usage unique) et applique un verrouillage du compte après cinq tentatives échouées. Les jetons JWT portent l'identité et le rôle ; le contrôle d'accès est appliqué à la fois côté serveur (middlewares) et côté client (garde de route).

5.2. Catalogue, panier et commande

Le catalogue et les fiches produits sont servis par l'API et consommés via React Query. Le panier est géré côté client (persistant) puis transformé en commande à l'étape de checkout : le serveur recalcule systématiquement les montants à partir des prix en base (jamais ceux du client) et décrémente le stock au sein d'une transaction, garantissant la cohérence.

5.3. Back-office administrateur

Le tableau de bord agrège le chiffre d'affaires, le nombre de commandes, la répartition des commandes par statut, les alertes de stock faible et l'évolution des ventes mensuelles. L'administrateur gère les produits (création, édition, suppression), fait évoluer le statut des commandes, active ou désactive les comptes, et consulte le journal d'activité.

5.4. Module d'Intelligence Artificielle

Le service de recommandation construit un prompt à partir de la fiche produit et de l'historique de l'utilisateur, interroge le LLM, valide la réponse, puis fait correspondre les suggestions à de vrais produits en base ; en l'absence de clé API, un filtrage par catégorie prend le relais. La génération de

fiches produits produit, à partir d'un nom et de mots-clés, un titre, une description marketing, des caractéristiques et des tags SEO, le résultat restant éditable avant enregistrement.

6. Sécurité applicative

Les exigences de sécurité du cahier des charges ont été traitées selon les catégories OWASP :

Menace (OWASP)	Mesure implémentée
Broken Access Control	JWT + middleware isAdmin sur chaque route ; garde de rôle côté front (ProtectedRoute) ; contrôle de propriété sur GET /orders/:id
Cryptographic Failures	Hachage bcrypt (hook beforeCreate) ; aucun mot de passe en clair
Injection	Requêtes paramétrées via l'ORM Sequelize
Security Misconfiguration	helmet, CORS, secrets en variables .env, HTTPS en production
Auth Failures	CAPTCHA au login + verrouillage après 5 tentatives + complexité MDP
Logging Failures	Table ActivityLog consultable dans le back-office

7. Tests, intégration continue et conteneurisation

La qualité est assurée par une suite de 14 tests unitaires Jest (services d'authentification, de produits et d'IA) exécutés avec couverture. Un pipeline GitHub Actions se déclenche à chaque push et pull request : vérification des types, tests back-end sur une base MySQL de test, build du front-end, audit de sécurité des dépendances, et construction des images Docker. Un job de déploiement conditionnel s'exécute uniquement sur la branche principale.

L'ensemble est conteneurisé via Docker Compose : trois services (MySQL, back-end, front-end) démarrent en une commande, les tables étant créées automatiquement et les données de démonstration injectées par un service dédié. L'environnement de démonstration est ainsi entièrement reproductible.

8. Périmètre implémenté et perspectives

Le périmètre implémenté (MVP) couvre le parcours e-commerce complet en paiement à la livraison, l'administration et les deux fonctionnalités d'IA. Certaines entités présentes dans la conception (avis clients, paiement en ligne, TVA détaillée, détection de fraude, analyse de sentiment) constituent des perspectives d'évolution non incluses dans le MVP.

Les évolutions envisagées incluent l'intégration d'un prestataire de paiement, la mise en cache des recommandations, les avis produits, la recherche à facettes, et le portage vers une application mobile native en réutilisant la couche de services.

9. Conclusion

ShopAI démontre la faisabilité d'une plateforme e-commerce complète, sécurisée et maintenable, dans laquelle l'Intelligence Artificielle résout des problèmes concrets : aider le client à découvrir des produits pertinents et assister l'administrateur dans la rédaction des fiches. Le projet met en œuvre une architecture en couches propre, des pratiques de sécurité conformes à l'OWASP, une chaîne d'intégration continue et une conteneurisation Docker, répondant ainsi aux exigences du cahier des charges.

10. Webographie

- Documentation React — react.dev

- Documentation Express — expressjs.com
- Documentation Sequelize — sequelize.org
- OWASP Top 10 — owasp.org
- Documentation Docker — docs.docker.com
- Documentation OpenAI API — platform.openai.com